



RDKS 胎壓感測器自動爬蟲系統

(每週自動更新本機版) · 本機環境手動執行 · 全自動化 · 中斷保護

核心特色

1. 極致 **SQL View** 壓縮：透過 SQL 原生的 GROUP_CONCAT 視圖，確保品牌、車系、型號相同的車款只會佔據一行 Excel。不同的感測器 (OE sensor)、頻率、HSN/TSN 會自動使用逗號整齊串接，版面極度乾淨！
2. 深度 **API** 解析：內建雙重 API 抓取。遇到隱藏的日期或頻率時，會自動解析 gpsr/data 接口補齊。
3. 無縫斷點接關與優雅中斷 (**Graceful Shutdown**)：隨時可以在 VS Code 按下暫停 (Ctrl+C 或停止鍵)。程式會攔截中斷訊號，將暫存區的殘留資料安全寫入資料庫並匯出備份，達成「零資料遺失」。下次啟動時，精準從中斷的「車系型號」接續努力，完美銜接零浪費。
4. 七天自動大掃除：內建七天週期檢查。只要距離上次完整抓取超過 7 天，按下啟動鍵時系統會自動失去記憶，進入「全面掃描模式」更新所有最新年份資料。
5. 智能環境適應 (突破 **PEP 668/uv** 限制)：執行時會自動檢查並安裝缺少的套件 (如 requests, pandas)。具備「三重突破機制」，即使在受保護的 Python 環境 (如 uv 管理的環境) 也能強制且安全地完成安裝，實現真正的一鍵啟動。

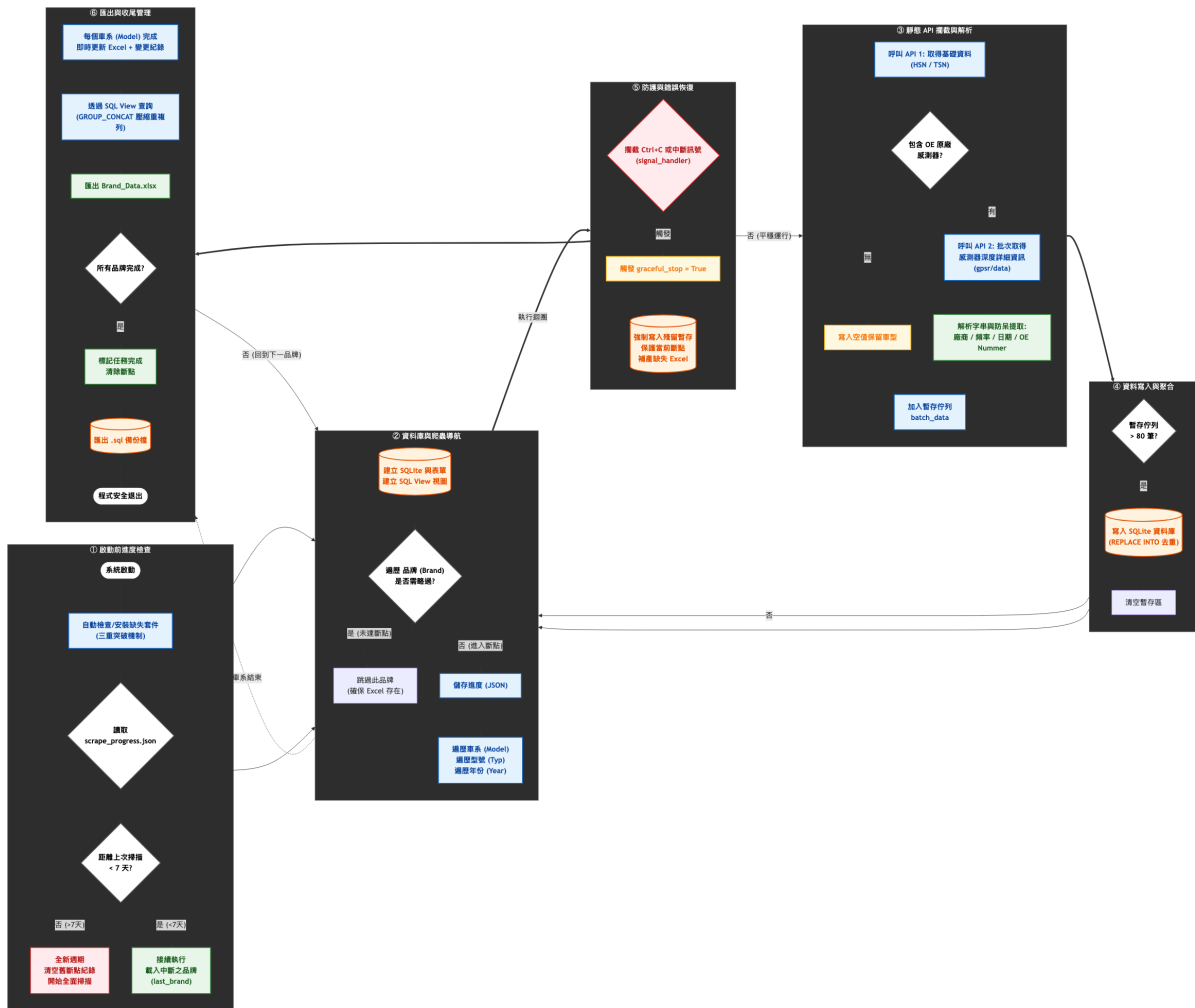
檔案產出說明

執行完畢後，所有最新資料會自動儲存於「胎壓資料庫_每週更新版/」資料夾內，並依據汽車品牌分類為獨立的 Excel 報表。

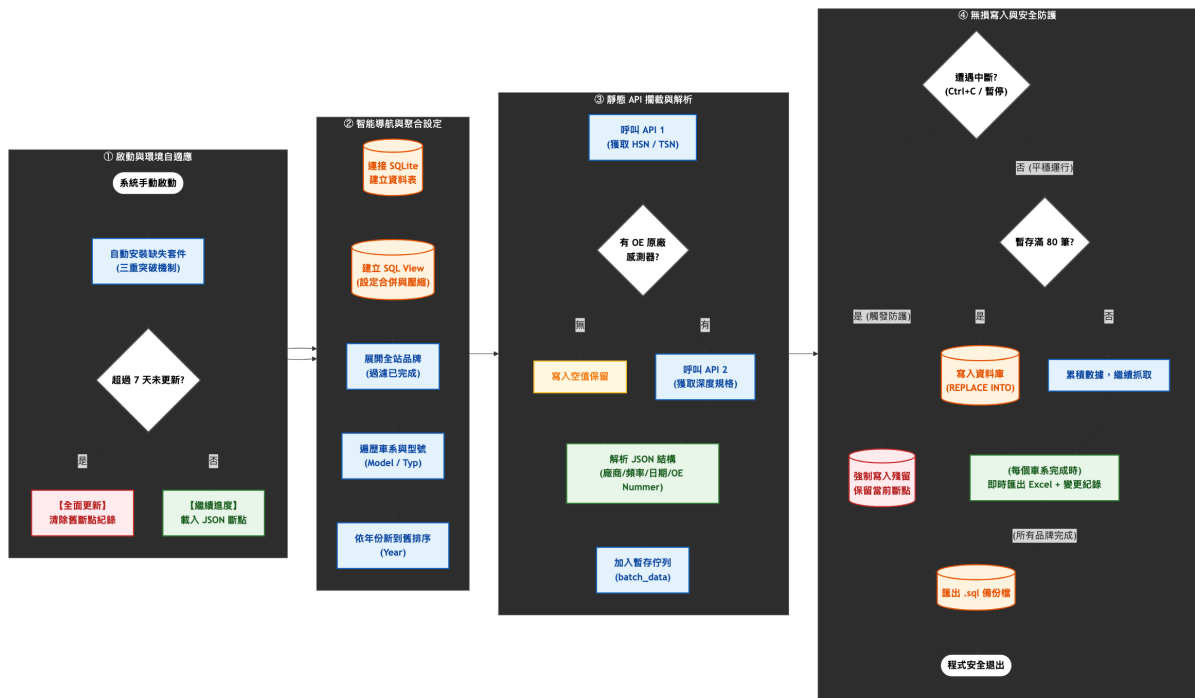
檔案	說明
{Brand}_Data.xlsx	各品牌最新資料 (即時更新)
變更紀錄.xlsx	變更報表 (新增🟢 / 修改🔴🟢 / 每日分頁)
胎壓檢測器資料庫_V10.db	SQLite 主資料庫
胎壓檢測器資料庫_V10_備份.db	DB 備份 (每次執行覆蓋)
胎壓檢測器資料庫_V10.sql	SQL dump 備份
scrape_progress_V10.json	斷點進度檔

詳細完整流程圖

RDKS 爬蟲系統核心架構圖 (Pro 專業版) — 完整收錄 7 天週期、API 雙軌抓取與優雅中斷防護的所有底層邏輯。



RDKS 爬蟲系統核心架構圖 (簡報專用版) — 系統抽象為四大平行階段，呈現 1x4 橫向佈局，最適合簡報提案。



📁 目錄結構

胎壓資料庫_每週更新版/

├─ {Brand}_Data.xlsx

└─ 變更紀錄.xlsx

程式根目錄：

├─ V10.py

├─ 胎壓檢測器資料庫_V10.db

├─ 胎壓檢測器資料庫_V10_備份.db

├─ 胎壓檢測器資料庫_V10.sql

└─ scrape_progress_V10.json



輸出檔案說明

1. 各品牌 Excel ({Brand}_Data.xlsx)

欄位	說明
Brand	品牌名稱
Model	型號分類
Typ	版本名稱
年份	生產年份區間
HSN	德國 HSN 碼
TSN	德國 TSN 碼
created date	感測器年份/批號
OE sensor	OE 感測器編號
OE Nummer	OE 零件號碼
Manufacturer	感測器製造商
Frequency	頻率 (433/434/315 MHz)

> 相同 品牌/車系/型號/年份 只佔一行，多個感測器以逗號串接。

2. 變更紀錄 (變更紀錄.xlsx)

- 每日一個分頁（格式 YYYYMMDD）
- 同一天中斷重啟 → 合併到同一頁
- 不同天 → 開新分頁
-  新增 (New) → 整行綠底綠字
-   修改 (Updated) → 舊值 (O) 紅底紅字、新值 (N) 綠底綠字
-  無變更 → 顯示「本日無任何變更」

中斷保護機制

情境	保護措施
Ctrl+C / 停止鍵	signal_handler 攔截 → graceful_stop=True → 安全寫入 殘留資料、保留斷點、補產 Excel
正常結束	寫入所有資料、Excel、變更紀錄、SQL 備份
程式崩潰	finally 區塊補產缺漏的 Excel + 寫入變更紀錄
強制終止 (SIGKILL)	DB 資料已即時寫入 (每 80 筆 / 每個車系)；下次啟動 recover_missing_excels() 自動補產
超過 7 天重跑	自動進入「全面掃描模式」，更新所有最新年份資料
中斷重啟 (同日)	變更紀錄合併到同一日分頁
中斷重啟 (隔日)	變更紀錄開新分頁

寫入時機 (確保即時性 / 零資料遺失)

資料	寫入時機
DB (SQLite)	暫存累積 80 筆；每個車系 (Model) 完成；中斷時 (Graceful Shutdown)；程式結束 finally
Brand Excel	每個車系完成 (即時更新)；啟動時 recover_missing_excels()；結束時 finally 補產
變更紀錄	每個車系完成 (即時更新)；結束時 finally 最終寫入
進度檔 JSON	每個 Brand 開始；每個 Type-Group

異常處理

- 缺少套件 → 啟動時自動偵測並安裝 (三重突破：pip → --user → --break-system-packages)
- API 請求失敗 → 自動重試 5 次 (backoff)，429 等待 10 秒
- DB 寫入失敗 → save_batch_to_sql 內含 try-except，不影響程式繼續
- Excel 寫入失敗 → _write_brand_diff 內含 try-except
- 無資料 → 感測器資料為空的 version 仍會寫入一筆空值佔位
- 中斷補產異常 → 印出警告，不影響主流程

執行方式

一鍵執行(前景): `python3 V10.py` - 自動安裝缺失套件

背景執行(推薦): `nohup python3 -u V10.py > run.log 2>&1 &`

查看進度: `tail -f run.log`

優雅中斷: `Ctrl+C` → 自動安全寫入殘留資料並儲存斷點，下次從原處續跑